

Learn Python for Automation

Lesson 8: Automating Tasks with Scheduling

Automating Tasks with Scheduling - Study Notes

Key Concepts

1. **The `schedule` library** – a lightweight, human readable way to schedule jobs in Python.
2. **Recurring jobs** – setting up tasks to run at specific times or intervals (e.g., daily, hourly).
3. **Integrating email notifications** – using `smtplib` (and optionally `email.mime`) to send alerts when a job finishes or fails.
4. **Graceful shutdown** – handling signals (e.g., `KeyboardInterrupt`) so the scheduler stops cleanly.
5. **Persisting state** – optional use of files or databases to keep track of job status across restarts.

Summary

Python's `schedule` library lets you declare tasks in a declarative style, making it easy to set up daily or hourly scripts without writing complex threading logic. A typical workflow involves defining a function that performs the desired work (e.g., data collection, report generation), scheduling it with `schedule.every().day.at("08:00").do(my\_job)`, and running a loop that calls `schedule.run\_pending()`.

To keep stakeholders informed, you can combine this with `smtplib` to send an email after each job runs. By constructing a MIME message, you can attach logs or results, set recipients, and send the message via an SMTP server (often requiring TLS and authentication). This pattern is common in automated reporting, monitoring, or maintenance scripts.

When deploying such scripts, consider signal handling to allow graceful termination, and optionally persist job status (e.g., last run timestamp) so you can detect missed runs after a crash or reboot.

Important Points

- **Installation**

```
```bash
pip install schedule
```
```

- **Basic scheduling syntax**

```
```python
import schedule, time

def job():
 print("Running daily task")
schedule.every().day.at("09:00").do(job)
while True:
 schedule.run_pending()
 time.sleep(1)
```
```

...

- **Email alert with `smtplib`**

```
```python
import smtplib
from email.mime.text import MIMEText
def send_email(subject, body, to_addr):
 msg = MIMEText(body)
 msg["Subject"] = subject
 msg["From"] = "sender@example.com"
 msg["To"] = to_addr
 with smtplib.SMTP("smtp.example.com", 587) as server:
 server.starttls()
 server.login("user", "pass")
 server.send_message(msg)
...

```

- **Combining job and email**

```
```python
def job():
    try:
        # perform work
        result = "Success!"
    except Exception as e:
        result = f"Failed: {e}"
    finally:
        send_email("Job Status", result, "admin@example.com")
...

```

- **Graceful shutdown**

```
```python
import signal
import sys
def signal_handler(sig, frame):
 print("Stopping scheduler...")
 sys.exit(0)
signal.signal(signal.SIGINT, signal_handler)
...

```

- **Persisting last run time** (simple example)

```
```python
import json, pathlib
STATUS_FILE = pathlib.Path("job_status.json")
def load_status():
    if STATUS_FILE.exists():
        return json.loads(STATUS_FILE.read_text())
...

```

```

    return {"last_run": None}
def save_status(status):
    STATUS_FILE.write_text(json.dumps(status))
...
---
```

Examples

1. Daily Backup Script with Success Email

```

```python
import schedule, time, subprocess, smtplib
from email.mime.text import MIMEText
def backup():
 try:
 # Example: tar archive of /data
 subprocess.check_call(["tar", "-czf", "/backups/data_$(date +%F).tar.gz", "/data"])
 status = "Backup completed successfully."
 except subprocess.CalledProcessError as e:
 status = f"Backup failed: {e}"
 finally:
 send_email("Daily Backup Status", status, "ops@example.com")
def send_email(subject, body, to_addr):
 msg = MIMEText(body)
 msg["Subject"] = subject
 msg["From"] = "backup@example.com"
 msg["To"] = to_addr
 with smtplib.SMTP("smtp.example.com", 587) as s:
 s.starttls()
 s.login("user", "pass")
 s.send_message(msg)
schedule.every().day.at("02:00").do(backup)
while True:
 schedule.run_pending()
 time.sleep(60)
...

```

**\*Explanation\*:** The script runs a shell command to create a compressed archive at 2 /AM every day. Whether it succeeds or fails, an email is sent to the operations team.

### 2. Hourly Health Check with Logging and Email Alert

```

```python
import schedule, time, requests, logging, smtplib
from email.mime.text import MIMEText
logging.basicConfig(filename="health.log", level=logging.INFO,

```

```

        format="%(%asctime)s %(levelname)s: %(message)s")
def health_check():
    try:
        r = requests.get("https://api.example.com/health", timeout=5)
        r.raise_for_status()
        status = f"Health OK: {r.json()}"
        logging.info(status)
    except Exception as e:
        status = f"Health check failed: {e}"
        logging.error(status)
        send_email("Health Check Failure", status, "devops@example.com")
def send_email(subject, body, to_addr):
    msg = MIMEText(body)
    msg["Subject"] = subject
    msg["From"] = "monitor@example.com"
    msg["To"] = to_addr
    with smtplib.SMTP("smtp.example.com", 587) as s:
        s.starttls()
        s.login("user", "pass")
        s.send_message(msg)
schedule.every(1).hour.do(health_check)
while True:
    schedule.run_pending()
    time.sleep(30)
...

```

***Explanation*:** Every hour, the script queries a health endpoint. On failure it logs an error and emails the DevOps team, ensuring rapid notification without manual monitoring.

Quick Review

1. What Python library provides a simple syntax for scheduling recurring jobs?
2. How do you schedule a job to run every day at 6:30 /PM using `schedule`?
3. Which Python module is used to send emails via SMTP?
4. Why is `starttls()` called before logging in to the SMTP server?
5. Name one way to make a scheduler stop gracefully when the user presses `Ctrl+C`.

Further Reading

- [Python `schedule` documentation](<https://schedule.readthedocs.io/>)
- [Python `smtplib` and `email` modules](<https://docs.python.org/3/library/smtplib.html>)
- Handling signals with `signal` and `asyncio` for more robust schedulers
- Using `APScheduler` for advanced scheduling (cron, job stores, thread pools)
- Securely storing SMTP credentials (e.g., environment variables, secret managers)

- Logging best practices with the `logging` module
- Building cron like jobs in cloud environments (AWS Lambda, Google Cloud Scheduler)

